

JENKINS-CHALLENGE

❖ Importance of Jenkins Pipeline

A Jenkins pipeline is a **structured way to automate CI/CD tasks** from source code to quality checks and notifications.

★ Why Jenkins Pipeline is Important

1. Automation

- Automatically pulls code, checks quality, and sends alerts without manual effort.

2. Consistency

- Every build follows the same steps, reducing human errors.

3. Faster Feedback

- Developers get quick feedback on code quality and build status.

4. Early Bug Detection

- Integrating SonarQube helps detect bugs, vulnerabilities, and code smells early.

5. Improved Collaboration

- Slack notifications instantly inform the team about build results.

6. Scalability

- Pipelines can be extended later to include Maven, Docker, deployment, etc.

7. Traceability

- Each stage (clone, scan, notify) is logged and auditable.

◆ Importance of Tools Used

■ 1. Jenkins (CI Tool)

Role: Orchestrator of the pipeline

Why Jenkins is important:

- Automates the CI workflow
- Supports thousands of plugins
- Easy integration with Git, SonarQube, Slack

- Open-source and widely used in industry
-

2. Git & GitHub

Role: Source Code Management

Importance:

- Centralized code repository
 - Version control and history tracking
 - Enables collaboration among developers
 - Jenkins pulls code directly from GitHub
-

3. SonarQube

Role: Static Code Quality & Security Analysis

Why SonarQube is important:

- Detects bugs, vulnerabilities, and code smells
 - Enforces coding standards
 - Improves maintainability and security
 - Helps maintain high code quality before deployment
-

4. PostgreSQL

Role: SonarQube Backend Database

Importance:

- Officially supported database for SonarQube
 - Reliable and high-performance
 - Stores metrics, issues, quality gates, and history
 - Better stability compared to deprecated MySQL support
-

5. Slack

Role: Notification & Collaboration Tool

Why Slack integration is useful:

- Sends real-time build and scan alerts
 - Improves team communication
 - Reduces dependency on Jenkins UI
 - Helps in faster issue resolution
-

6. Docker (Used during troubleshooting)

Role: Alternative deployment option

Importance:

- Provides quick setup
 - Avoids dependency conflicts
 - Helps compare containerized vs traditional setup
 - Useful for learning and fallback deployment
-

7. Java (JDK 17)

Role: Runtime for Jenkins & SonarQube

Importance:

- Required for Jenkins and SonarQube execution
 - Correct Java version prevents compatibility issues
 - Ensures Elasticsearch and Sonar services run properly
-

Overall Architecture Importance

Developer → GitHub → Jenkins Pipeline



SonarQube Analysis



Slack Notification

Use the below code and implement our CI/CD stages.

- **Source Code:** <https://github.com/betawins/hiring-app.git>
 - **Repository contains the application code for the hiring app.**
- **Create one Declarative pipeline job**
- **Create one Scripted pipeline job**
- **Create one multi stage pipeline job**
- **Create one parallel stage pipeline job**

Jenkins Official Documentation:

- Go to master Jenkins instance start the instance
- After starting the instance go to inside instance connect with ssh
- After connecting start Jenkins server
- And then open Jenkins GUI server

```

[ec2-user@ip-172-31-21-117 ~]$ systemctl start maven
Failed to start maven.service: Unit maven.service not found.
See system logs and 'systemctl status maven.service' for details.
[ec2-user@ip-172-31-21-117 ~]$ sudo su
Last login: Fri Jan 2 13:14:34 UTC 2026 on pts/1
Failed to start maven.service: Unit maven.service not found.
[ec2-user@ip-172-31-21-117 ~]$ systemctl start maven
Failed to start maven.service: Unit maven.service not found.
[ec2-user@ip-172-31-21-117 ~]$ sudo -v
Apache Maven 3.9.4 (Red Hat 3.9.4-3.mnsm0223.0.5)
Home: /usr/share/maven
Java version: 17.0.17, vendor: Amazon.com Inc., runtime: /usr/lib/jvm/java-17-amazon-corretto.x86_64
OS name: "linux", version: "4.12.58-62.el1.amzn2023.x86_64", arch: "amd64", family: "unix"
[ec2-user@ip-172-31-21-117 ~]$ cd /opt
[ec2-user@ip-172-31-21-117 ~]$ ls -ld /opt
drwxr-xr-x 1 root root 4096 Nov 12 2025 /opt
$ mv ./maven-3.9.4-3.0.2.0470-linux-slp amazon_scmanner
$ mv ./maven-3.9.4-3.0.2.0470-linux-slp opt/$ mv amazon_scmanner/
$ cd conf $rm *.log
[ec2-user@ip-172-31-21-117 amazon_scmanner]$ cd conf/
[ec2-user@ip-172-31-21-117 conf]$ mv amazon_scmanner.properties
[ec2-user@ip-172-31-21-117 conf]$ █

```

- Go to nexus instance and open instance and connect with ssh-i
- After connecting start nexus and open in browser with 8081 port number

```

      ^
East login: Fri Jan  2 07:02:33 2026 from 18.204.107.29
[ec2-user@ip-172-31-84-50 ~]$ systemctl start nexus
Authentication is required to start 'nexus.service'.
Authentication as: EC2 Default User (ec2-user)
Password:
sh: /usr/bin/expect-helper: line 1: pam_authenticate failed: Authentication failure

AUTHENTICATION FAILED =====
Failed to start nexus.service: Access denied
See system logs and 'systemctl status nexus.service' for details.
[ec2-user@ip-172-31-84-50 ~]$ sudo su -
East login: Fri Jan  2 07:02:43 UTC 2026 on pts/1
[root@ip-172-31-84-50 ~]# systemctl start nexus
[root@ip-172-31-84-50 ~]# systemctl status nexus
nexus.service - nexus.service
Loaded: loaded (/etc/systemd/system/nexus.service; enabled; preset: disabled)
Active: active (running) since Mon 2026-01-05 06:52:53 UTC 27min ago
Process: 2518 ExecStart=/opt/nexus/bin/nexus start (code=exited, status=0/SUCCESS)
Main PID: 2791 (java)
Tasks: 85 (limit: 9492)
Memory: 2.2G
CPU: 1min 37.06s
CGroup: /system.slice/nexus.service
        └─2791 /usr/lib/jvm/java/bin/java -server -Dinstall4j.jvmDir=/usr/lib/jvm/java -Dexe4j.moduleName=/opt/nexus/bin/nexus -XX:+UnlockDiagnosticVMOptions -Dinstall4j.launcherId=245 -Dinstall4j.nrc=false -Dlog=yv=0

Jan 05 06:52:53 ip-172-31-84-50.ec2.internal systemd[1]: Starting nexus.service - nexus.service...
Jan 05 06:52:53 ip-172-31-84-50.ec2.internal systemd[1]: Starting nexus.service - nexus.service...
Jan 05 06:52:53 ip-172-31-84-50.ec2.internal systemd[1]: Starting nexus.service - nexus.service...
Jan 05 06:52:53 ip-172-31-84-50.ec2.internal systemd[1]: Starting nexus.service - nexus.service...
lines 1-1/14 (END)

```

- Go to sonarqube server open and connect
- Start docker and start sonarqube

```
last login: Fri Jan 2 06:13:32 2026 from 10.206.107.29
[ec2-user@ip-172-31-92-107 ~]$ sudo su -
root: sshd command not found
[ec2-user@ip-172-31-92-107 ~]$ sudo su -
last login: Fri Jan 2 06:14:18 UTC 2026 on pts/1
[ec2-user@ip-172-31-92-107 ~]$ systemctl start docker
[ec2-user@ip-172-31-92-107 ~]$ systemctl status docker
* docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-01-05 06:12:44 UTC; 23min ago
   TriggeredBy: * docker.socket
   Docs: https://docs.docker.com
   Process: 2067 ExecStartPre=/usr/bin/mkdir -p /run/docker (code=exited, status=0/SUCCESS)
   Process: 2071 ExecStartFree=/usr/libexec/docker/docker-setup-runtime.sh (code=exited, status=0/SUCCESS)
   Main PID: 2072 (dockerd)
   Tasks: 0
   Memory: 103.1M
   CPU: 790ms
   CGroup: /systemd.slice/docker.service
           └─2072 /usr/bin/dockerd -B fd:// --containerd=/run/containerd/containerd.sock --default-ulimit nofile=32768:65536

Jan 05 06:12:42 ip-172-31-92-107.ec2.internal systemd[1]: Starting docker.service - Docker Application Container Engine...
Jan 05 06:12:42 ip-172-31-92-107.ec2.internal dockerd[2072]: time="2026-01-05T06:12:42.851099462Z" level=info msg="Starting up"
Jan 05 06:12:43 ip-172-31-92-107.ec2.internal dockerd[2072]: time="2026-01-05T06:12:43.046371382Z" level=info msg="(graphdriver) using prior storage driver: overlay2"
Jan 05 06:12:43 ip-172-31-92-107.ec2.internal dockerd[2072]: time="2026-01-05T06:12:43.093426627Z" level=info msg="Loading containers: start."
Jan 05 06:12:43 ip-172-31-92-107.ec2.internal dockerd[2072]: time="2026-01-05T06:12:43.840965236Z" level=info msg="Default bridge (docker0) is assigned with an IP address 172.17.0.0/16. Daemon option --bip can be used to set a preferred IP address"
Jan 05 06:12:43 ip-172-31-92-107.ec2.internal dockerd[2072]: time="2026-01-05T06:12:43.847891182Z" level=info msg="Loading containers: done."
Jan 05 06:12:44 ip-172-31-92-107.ec2.internal dockerd[2072]: time="2026-01-05T06:12:44.017623341Z" level=info msg="Docker daemon" commit=16551fe containerd-snapshotter=false storage-driver=overlay2 version=25.0.13
Jan 05 06:12:44 ip-172-31-92-107.ec2.internal dockerd[2072]: time="2026-01-05T06:12:44.017759991Z" level=info msg="Daemon has completed initialization"
Jan 05 06:12:44 ip-172-31-92-107.ec2.internal dockerd[2072]: time="2026-01-05T06:12:44.034446792Z" level=info msg="API listen on /run/docker.sock"
Jan 05 06:12:44 ip-172-31-92-107.ec2.internal systemd[1]: Started docker.service - Docker Application Container Engine.
[ec2-user@ip-172-31-92-107 ~]$ docker ps
CONTAINER ID        IMAGE               COMMAND                  CREATED        STATUS        PORTS                    NAMES
2ab294eb5f6c        sonarqube/sonarq... "/opt/sonarqube/doch...  4 days ago     Up 5 seconds   0.0.0.0:9000->9000/tcp, :::9000->9000/tcp   sonarqube
```

- Create new job for declarative job with selecting pipeline

New Item

Enter an item name

declarative_pipeline2

Select an item type

Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

Maven project

Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.

Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.

Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

Multibranch Pipeline

Creates a set of Pipeline projects according to detected branches in one SCM repository.

OK

- Update the URL as it is changed when we closed /opt/sonar-scannar/con/sonar-properties

```
Configure here general information about the environment, such as SonarQube server connection details for example
Info: Information about specific project should appear here.

#----- Default SonarQube server
sonar.host.url=http://34.227.18.237:9000

#----- Default source code encoding
#sonar.sourceEncoding=UTF-8
```

- Update in tool configure as well

SonarQube installations

List of SonarQube installations

Name

SonarQube

Server URL

Default is http://localhost:9000

http://34.227.18.237:9000

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

jenkins-pipeline

+ Add

Advanced

```
pipeline {
    agent any

    environment {
        // SonarQube
        SONAR_SCANNER = tool 'sonar_scanar'
        SONAR_PROJECT_KEY = 'hiring-app'
        SONAR_PROJECT_NAME = 'hiring-app'

        // Nexus
        NEXUS_URL = 'http://3.95.149.225:8081'
        NEXUS_REPO = 'hiring_Maven_Webapp'
        NEXUS_CRED = 'Nexus_server'

        // Tomcat
        TOMCAT_URL = 'http://100.27.210.128:8080'
        TOMCAT_CONTEXT = 'hiring'
        TOMCAT_CRED = 'tomcat-deploy'
    }

    stages {

        stage('Git Clone') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/betawins/hiring-app.git'
            }
        }
    }
}
```

```
}
```

```
stage('Maven Build') {  
    steps {  
        sh 'mvn clean package -DskipTests'  
    }  
}
```

```
stage('SonarQube Analysis') {  
    steps {  
        withSonarQubeEnv('SonarQube') {  
            sh '''  
                ${SONAR_SCANNER}/bin/sonar-scanner \  
                -Dsonar.projectKey=${SONAR_PROJECT_KEY} \  
                -Dsonar.projectName=${SONAR_PROJECT_NAME} \  
                -Dsonar.sources=src \  
                -Dsonar.java.binaries=target  
            '''  
        }  
    }  
}
```

```
stage('Upload WAR to Nexus') {  
    steps {  
        withCredentials([usernamePassword(  
            credentialsId: "${NEXUS_CRED}",  
            usernameVariable: 'NEXUS_USER',  
            passwordVariable: 'NEXUS_PASS'
```

```

    )) {
        sh ""

        WAR_FILE=$(ls target/*.war | head-n 1)

        curl-u $NEXUS_USER:$NEXUS_PASS \
        --upload-file $WAR_FILE \
        ${NEXUS_URL}/repository/${NEXUS_REPO}/
    ""
    }
}
}

```

```

stage('Deploy to Tomcat') {

```

```

    steps {
        withCredentials([usernamePassword(
            credentialsId: "${TOMCAT_CRED}",
            usernameVariable: 'TOMCAT_USER',
            passwordVariable: 'TOMCAT_PASS'

```

```

        )) {

```

```

            sh ""

            WAR_FILE=$(ls target/*.war | head-n 1)

            curl-u $TOMCAT_USER:$TOMCAT_PASS \
            -T $WAR_FILE \

```

```

            "${TOMCAT_URL}/manager/text/deploy?path=/${TOMCAT_CONTEXT}&update=true"

```

```

        ""

```

```

    }

```



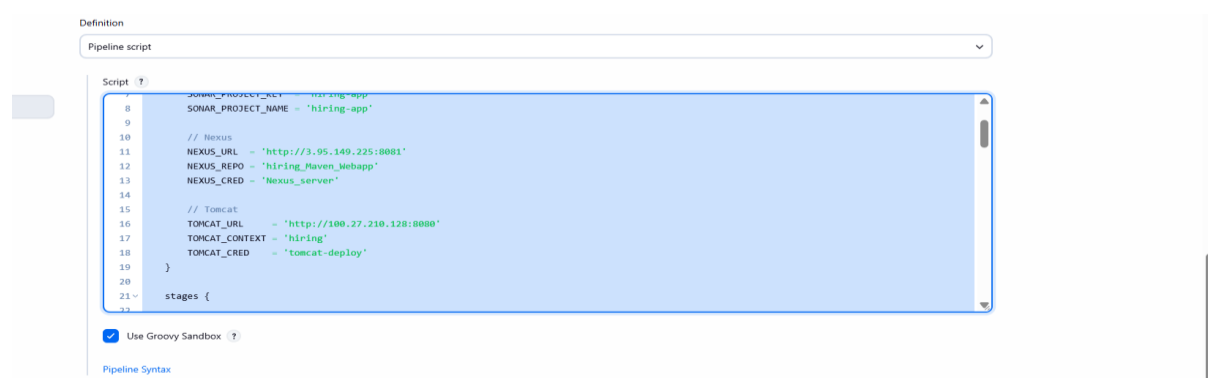
```

    }
}
}

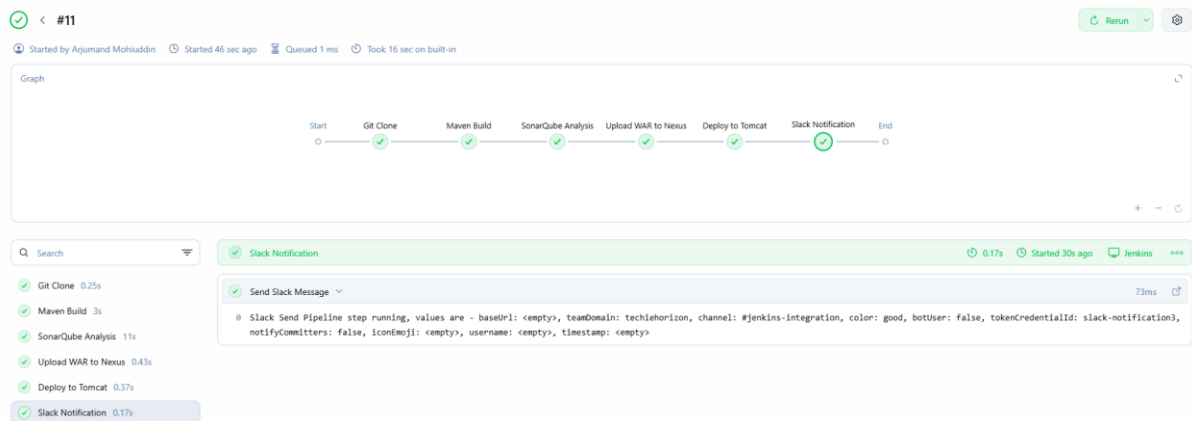
post {
    success {
        slackSend(
            channel: '#jenkins-integration',
            color: 'good',
            message: "✅ SUCCESS — ${JOB_NAME} #${BUILD_NUMBER}"
        )
    }
    failure {
        slackSend(
            channel: '#jenkins-integration',
            color: 'danger',
            message: "❌ FAILED — ${JOB_NAME} #${BUILD_NUMBER}"
        )
    }
}
}
}

```

Pipeline which I have used



Build was executed successfully



2. Create one Scripted pipeline job

Go to dashboard and create a new job for scripted

New Item

Enter an item name

Scripted2

Select an item type

- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
- Multibranch Pipeline**
Creates a set of Pipeline projects according to detected branches in one SCM repository.

OK

Add the script in the pipeline

```
node {  
    try {  
        stage('Git Clone') {  
            git branch: 'main',  
            url: 'https://github.com/betawins/hiring-app.git'  
        }  
  
        stage('Build') {
```

```
    sh 'mvn clean package -DskipTests'
}

stage('SonarQube Analysis') {
    withSonarQubeEnv('SonarQube') {
        sh 'mvn sonar:sonar'
    }
}

stage('Deploy') {
    sh 'echo "Deploying application..."'
}

slackSend(
    channel: '#jenkins-integration',
    color: 'good',
    message: "✅ SUCCESS — ${JOB_NAME} #${BUILD_NUMBER}"
)

} catch (err) {

    slackSend(
        channel: '#jenkins-integration',
        color: 'danger',
        message: "❌ FAILED — ${JOB_NAME} #${BUILD_NUMBER}"
    )

    throw err
}
```

```
}  
  
}
```

Configure

- General
- Triggers
- Pipeline
- Advanced

Definition

Pipeline script

Script ?

```
25 message: "✅ SUCCESS - ${JOB_NAME} #${BUILD_NUMBER}"  
26 )  
27  
28 } catch (err) {  
29  
30 // FAILURE SLACK  
31 slackSend(  
32 channel: '#jenkins-integration',  
33 color: 'danger',  
34 message: "❌ FAILED - ${JOB_NAME} #${BUILD_NUMBER}"  
35 )  
36 throw err  
37 }  
38 }  
39 }
```

try sample Pipeline...

☒ Use Groovy Sandbox ?

Pipeline Syntax

Advanced

Advanced

Save

Apply

Job was built

Jenkins / Scripted2 / #3 / Stages

Started by Ajumand Mohiuddin Started 1 min 0 sec ago Queued 1 ms Took 18 sec on built-in

Graph

Start → Git Clone → Build → SonarQube Analysis → Deploy → End

Deploy 0.28s Started 42s ago Jenkins

echo "Deploying application..."
+ echo "Deploying application..."
1 Deploying application...

3. Create one multi stage pipeline job

Create a new job for multi stages

New Item

Enter an item name

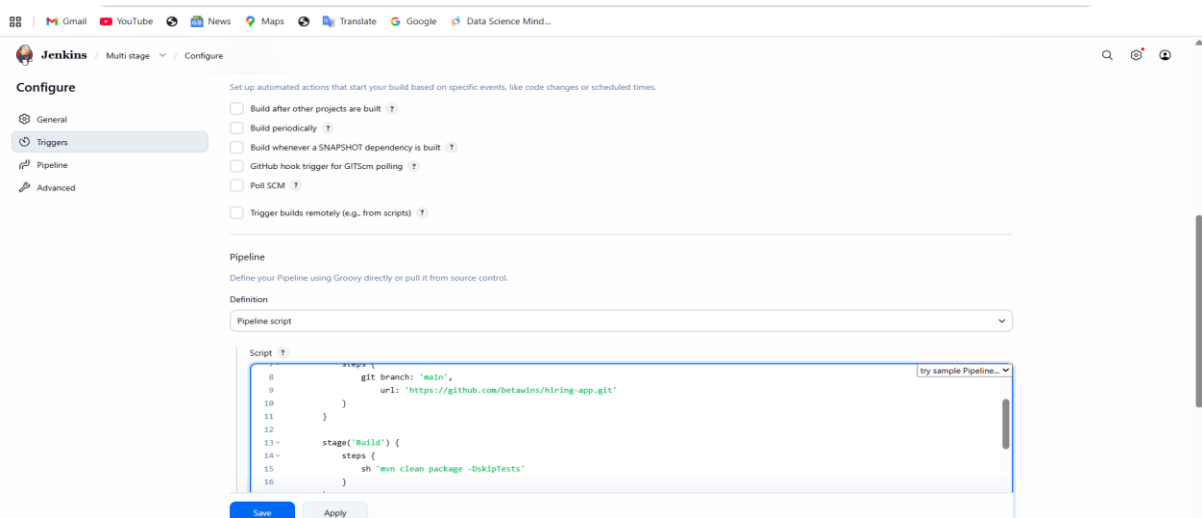
Multi stage

Select an item type

- Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Pipeline
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
- Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
- Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.

OK

Add script for multi stages build



Following script was executed for multi stages pipeline

```
pipeline {
```

```
    agent any
```

```
    stages {
```

```
        stage('Git Clone') {
```

```
            steps {
```

```
                git branch: 'main',
```

```
                url: 'https://github.com/betawins/hiring-app.git'
```

```
            }
```

```
        }
```

```
        stage('Build') {
```

```
            steps {
```

```
                sh 'mvn clean package -DskipTests'
```

```
            }
```

```
        }
```

```
stage('SonarQube Analysis') {  
    steps {  
        withSonarQubeEnv('SonarQube') {  
            sh 'mvn sonar:sonar'  
        }  
    }  
}
```

```
stage('Deploy') {  
    steps {  
        sh 'echo "Deploying application..."'  
    }  
}
```

```
post {  
    success {  
        slackSend(  
            channel: '#jenkins-integration',  
            color: 'good',  
            message: "✅ SUCCESS — ${JOB_NAME} #${BUILD_NUMBER}"  
        )  
    }  
    failure {  
        slackSend(  
            channel: '#jenkins-integration',  
            color: 'danger',  
            message: "❌ FAILED — ${JOB_NAME} #${BUILD_NUMBER}"  
        )  
    }  
}
```

```

    )
}

}

}

```

Jenkins / Multi stage / #1 / Stages

#1

Started by Ajumand Mohiuddin Started 1 min 1 sec ago Queued 2 ms Took 17 sec on built-in

Graph

Start Git Clone Build SonarQube Analysis Deploy Post Actions End

Search

Git Clone 0.29s

Build 4s

SonarQube Analysis 12s

Deploy 0.33s

Post Actions 0.11s

Post Actions 0.11s Started 44s ago Jenkins

Send Slack Message 76ms

Slack Send Pipeline step running, values are - baseUrl: <empty>, teamDomain: techlehorizon, channel: #jenkins-integration, color: good, botUser: false, tokenCredentialId: slack-notification3, notifyCommiters: false, iconEmoji: <empty>, username: <empty>, timestamp: <empty>

4. Create one parallel stage pipeline job

Go to dashboard and create a new job for parallel pipeline

New Item

Enter an item name

parallel

Select an item type

- Freestyle project**
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- Maven project**
Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.
- Pipeline**
Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.
- Multi-configuration project**
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- Folder**
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
- Multibranch Pipeline**
Creates a set of Pipeline projects according to detected branches in one SCM repository.

OK

Following script I have used

```

pipeline {
    agent any

    environment {
        SLACK_CHANNEL = '#jenkins-integration'
    }
}

```

```
}
```

```
stages {
```

```
  stage('Git Clone') {
```

```
    steps {
```

```
      git branch: 'main',
```

```
      url: 'https://github.com/betawins/hiring-app.git'
```

```
    }
```

```
  }
```

```
  stage('Parallel Tasks') {
```

```
    parallel {
```

```
      stage('Build') {
```

```
        steps {
```

```
          sh 'mvn clean package -DskipTests'
```

```
        }
```

```
      }
```

```
      stage('SonarQube Scan') {
```

```
        steps {
```

```
          withSonarQubeEnv('SonarQube') {
```

```
            sh 'mvn sonar:sonar'
```

```
          }
```

```
        }
```

```
      }
```

```
    }
```



```
}
```

```
stage('Deploy') {  
    steps {  
        sh 'echo "Deploying application..."'  
    }  
}
```

```
/* 🛎️ VISIBLE SLACK STAGE */
```

```
stage('Slack Notification') {  
    steps {  
        script {  
            if (currentBuild.currentResult == 'SUCCESS') {  
                slackSend(  
                    channel: "${SLACK_CHANNEL}",  
                    color: 'good',  
                    message: "✅ SUCCESS — ${JOB_NAME} #${BUILD_NUMBER}"  
                )  
            } else {  
                slackSend(  
                    channel: "${SLACK_CHANNEL}",  
                    color: 'danger',  
                    message: "❌ FAILED — ${JOB_NAME} #${BUILD_NUMBER}"  
                )  
            }  
        }  
    }  
}
```

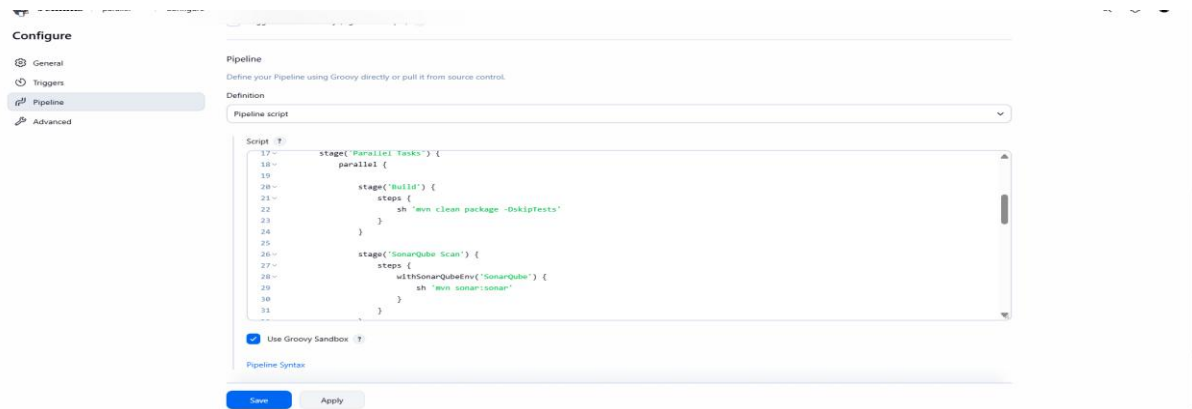
```

}

/* 🛡️ SAFETY NET */

post {
    failure {
        slackSend(
            channel: "${SLACK_CHANNEL}",
            color: 'danger',
            message: "🚨 PIPELINE FAILED — ${JOB_NAME} #${BUILD_NUMBER}"
        )
    }
}
}
}

```



Job was built successfully

